```
end
```

Usually, this is perfectly good enough—you associate a fixed block of code with a method in the same way you'd have a chunk of code after an if or while statement.

But sometimes you'd like to be more flexible. In this example, we're teaching math skills. The student could ask for an *n*-plus table or an *n*-times table. If the student asked for a 2-times table, we'd output 2, 4, 6, 8, and so on. (This code does not check its inputs for errors.)

```
print "(t)imes or (p)lus: "
operator = gets
print "number: "
number = Integer(gets)

if operator.start_with?("t")
  puts((1..10).collect { |n| n*number }.join(", "))
else
  puts((1..10).collect { |n| n+number }.join(", "))
end
```

produces:

```
(t)imes or (p)lus: t
number: 2
2, 4, 6, 8, 10, 12, 14, 16, 18, 20
```

This works, but it's ugly, with virtually identical code on each branch of the if statement. It would be nice if we could factor out the block that does the calculation:

```
print "(t)imes or (p)lus: "
operator = gets
print "number: "
number = Integer(gets)
if operator.start_with?("t")
  calc = -> (n) { n * number }
else
  calc = -> (n) { n + number }
end
puts((1..10).map(&calc).join(", "))
```

produces:

```
(t)imes or (p)lus: t
number: 2
2, 4, 6, 8, 10, 12, 14, 16, 18, 20
```

In this version, we assign the correct block to a variable named calc, and then in the highlighted line, we pass calc to the standard method map, prefixing it with an &, (&calc).

If the last argument to a method is preceded by an ampersand, Ruby calls to_proc on the object. It removes it from the argument list, converts the Proc object into a block, and associates it with the method. In this case, the object is already a Proc, so that means that map is called with the lambda as its block argument and uses that block to convert the elements of the method receiver.

There's a shorter way to write this code. Ruby objects have a method named method, which takes a symbol and returns the object's method of the same name. We can use the same to_proc feature that symbols have: